A

Project Report on

Implementing BLE Anomaly Detection on Raspberry Pi for IoT Security

Submitted by:

K Anirudh
K Sreedhar

Under the Guidance of

Dr. P Ranga Babu Sir

Associate Professor

In Partial Fulfilment of the Requirement of the Subject in BE (Electronics and Communication Engineering) Programme

Submitted to,

(IIITDM Kurnool Kurnool)

Academic Year- (2025)

Abstract

This project investigates the implementation of anomaly detection in Bluetooth Low Energy (BLE) communications using Raspberry Pi and ESP32 microcontrollers for IoT security applications. A wireless sensor network was created with multiple ESP32 devices, each interfaced with environmental sensors such as ultrasonic, IR, touch, and DHT sensors. These ESP32s periodically broadcast sensor data using BLE. The central Raspberry Pi node was programmed to automatically discover, connect, and rotate connections among all nearby ESP32 devices, maintaining a connection for 120 seconds per device. The primary objective was to extract critical BLE advertisement features—including MAC address, device name, RSSI, flags, packet length, UUID, and inter-arrival time (IAT)—to support robust anomaly detection. The project evaluates several BLE sniffing and logging tools, highlights practical challenges, and demonstrates a complete workflow for secure and automated BLE feature collection and analysis in IoT environments.

INTRODUCTION

The increasing integration of IoT devices into daily life has introduced both new opportunities and novel security risks. Bluetooth Low Energy (BLE) is especially popular for its energy efficiency, cost-effectiveness, and ease of deployment, making it ubiquitous in smart homes, industrial sensors, medical devices, and consumer electronics. However, BLE's reliance on open, broadcast-based advertising for device discovery also introduces vulnerabilities—malicious devices can impersonate legitimate devices, launch denial-of-service attacks, or exfiltrate data.

Traditional BLE security solutions often focus on authentication and encryption for connected sessions, but these do not address threats that arise in the advertising channel itself. **Anomaly detection**, based on the analysis of broadcast packet features, can serve as an early warning mechanism for the presence of rogue or compromised devices.

This project explores a practical, hands-on approach to BLE anomaly detection using commodity hardware. The core network comprises multiple ESP32 microcontrollers, each wired to a set of environmental sensors (ultrasonic, IR, touch, DHT). These ESP32s are arranged in a mesh or star topology and programmed to broadcast their sensor data via BLE advertising packets. A central Raspberry Pi node is tasked with the automatic and dynamic discovery of all nearby ESP32s, managing connections by spending 120 seconds with each device before rotating to the next.

The main objective is to **collect rich BLE advertisement data**—specifically, MAC address, device name, RSSI, flags, packet length, UUID, and IAT—and to build a pipeline that can extract, store, and analyse these features for the detection of abnormal behaviour or unauthorized devices in the BLE environment.

METHODOLOGY

System Overview

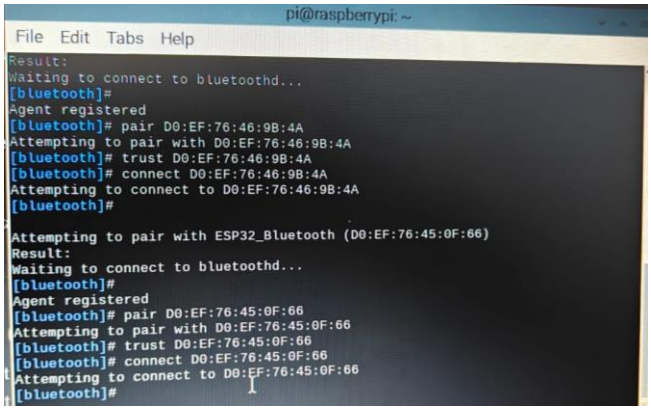
The system architecture consists of multiple sensor-equipped ESP32s and a Raspberry Pi central node. Each ESP32 collects real-world data via sensors and periodically broadcasts this information using BLE advertisements. The Raspberry Pi, running Linux and equipped with a Bluetooth adapter, operates as the BLE master or “sniffer.”

ESP32 Sensor Network

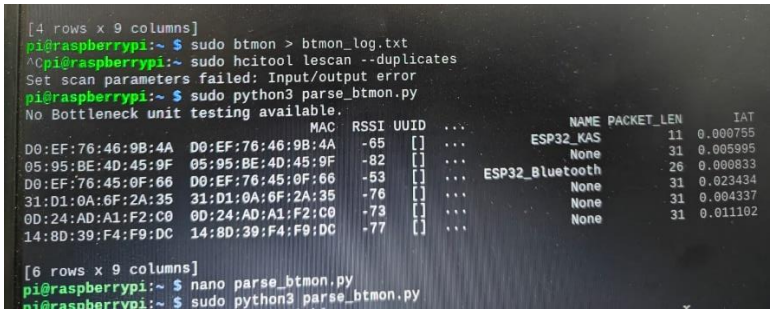
- **Sensors Used:** Ultrasonic sensors (for distance), IR sensors (for object detection), touch sensors, and DHT sensors (for temperature and humidity).
- **Broadcast Topology:** ESP32s are placed throughout the environment in a mesh or star configuration to simulate a real-world deployment.

Raspberry Pi BLE Collector

- **Automatic Connection Logic:** A custom Python and bash-based script scans for all nearby BLE devices, identifies ESP32s based on their advertising data, and connects sequentially to each for a 120-second interval, rotating to ensure all devices are polled in turn.



- **Motivation for Automation:** Manual pairing and data collection would be infeasible for large or dynamic sensor networks; hence, the process is fully automated.



Data Collection and Feature Extraction

The primary objective was to extract features necessary for anomaly detection:
MAC address, device name, RSSI, flags, packet length, UUID, and IAT.

Tool	Outcome	Notes
Bluepy	Not suitable	Limited advertisement feature extraction (RSSI/UUID/Flags often zero)
Scapy	Not suitable	Good for BLE pentesting but failed to capture required advertisement fields in this use-case
BlueZ	Not suitable	Standard Linux BLE stack but unable to provide rich packet-level data needed
btmon	Suitable	Captured complete HCI event logs, including all advertising packet details

IMPLEMENTATION

Hardware Setup

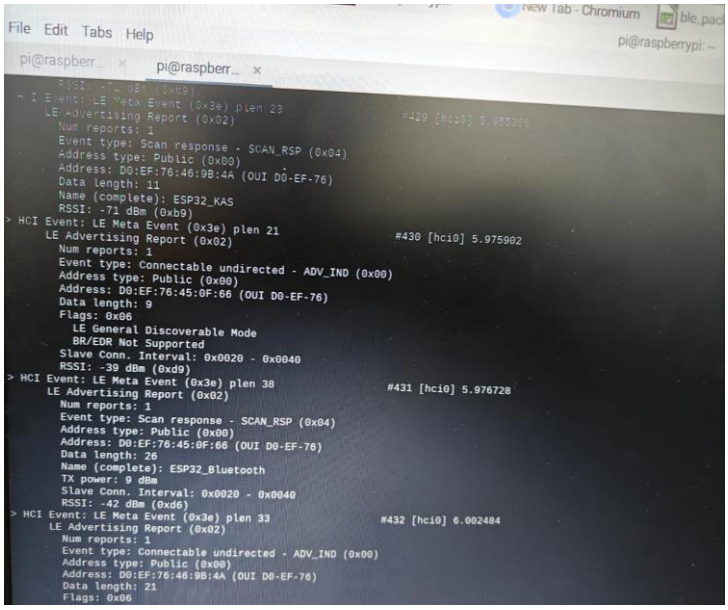
- **ESP32s:** Each connected to a set of sensors, configured to periodically broadcast data using BLE advertisements.
- **Raspberry Pi:** Acts as the BLE scanner and collector, running Linux and required BLE tools.

Software Workflow

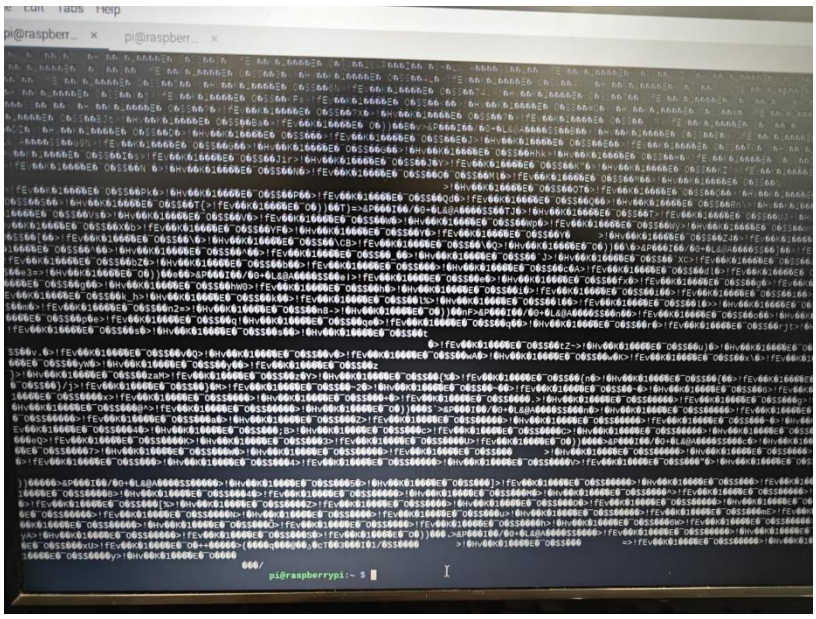
1. **BLE Advertisement:** ESP32s broadcast data in BLE advertising packets.
2. **Automatic Device Discovery:** The Raspberry Pi is programmed (using Python and bash scripts) to scan for all nearby BLE devices, identify ESP32s, and connect in a rotating fashion for 120 seconds each.
3. **Packet Capture:** btmon runs continuously on the Raspberry Pi, capturing all BLE HCI events (including advertisements, scan responses, and connection events).
4. **Packet Parsing:** PCAP files are decrypted and parsed using custom Python scripts to extract MAC, device name, RSSI, flags, packet length, UUID, and IAT.
5. **Data Storage:** Extracted features are stored in CSV/Excel format for analysis.

Technical Challenges and Solutions

- **Incomplete Feature Capture:** Initial attempts with Bluepy and Scapy failed to yield complete BLE advertisement data, particularly for RSSI, UUID, and flags.
- **BLE Tool Limitations:** Discovering that only advertisement sniffing (not connection mode) provides full feature data led to the switch to btmon.
- **Automation:** Implementing automatic rotation among ESP32s ensured fair and comprehensive data collection.



Encrypted Packet



RESULTS AND DISCUSSION

Feature Extraction

Using the final pipeline, the following features were successfully extracted for each BLE packet:

- **MAC Address:** Unique hardware identifier of each ESP32.
- **Device Name:** Human-readable name for easier identification.
- **RSSI:** Signal strength, useful for proximity estimation and anomaly detection.
- **Flags:** Advertisement flags indicating BLE mode.
- **Packet Length:** Size of the BLE advertisement packet.
- **UUID:** Service identifier for distinguishing device functions.
- **IAT (Inter-Arrival Time):** Time between consecutive packets, useful for behavioral analysis.

THE FINAL OUTPUT:

	A	B	C	D	E	F	G	H	I
1	Event Type	MAC	Name	RSSI	TX Power	Conn. Interval	Packet Len	Flags	UUID
2	Scan response - SCAN_RSP	D0:EF:76:45:0F:66	ESP32_Bluetooth	-38	9	0x0020 - 0x0040	26		
3	Connectable undirected - ADV_IND	D0:EF:76:46:9B:4A		-64			21	0x06	4fafc201-1fb5-459e-8fcc-c5c9c331914b
4	Scan response - SCAN_RSP	D0:EF:76:46:9B:4A	ESP32_KAS	-65			11		
5	Connectable undirected - ADV_IND	D0:EF:76:45:0F:66		-42		0x0020 - 0x0040	9	0x06	
6	Scan response - SCAN_RSP	D0:EF:76:45:0F:66	ESP32_Bluetooth	-42	9	0x0020 - 0x0040	26		
7	Connectable undirected - ADV_IND	D0:EF:76:46:9B:4A		-71			21	0x06	4fafc201-1fb5-459e-8fcc-c5c9c331914b
8	Scan response - SCAN_RSP	D0:EF:76:46:9B:4A	ESP32_KAS	-70			11		
9	Connectable undirected - ADV_IND	D0:EF:76:45:0F:66		-44		0x0020 - 0x0040	9	0x06	
10	Scan response - SCAN_RSP	D0:EF:76:45:0F:66	ESP32_Bluetooth	-44	9	0x0020 - 0x0040	26		
11	Connectable undirected - ADV_IND	D0:EF:76:46:9B:4A		-64			21	0x06	4fafc201-1fb5-459e-8fcc-c5c9c331914b
12	Scan response - SCAN_RSP	D0:EF:76:46:9B:4A	ESP32_KAS	-64			11		
13	Connectable undirected - ADV_IND	D0:EF:76:45:0F:66		-42		0x0020 - 0x0040	9	0x06	
14	Scan response - SCAN_RSP	D0:EF:76:45:0F:66	ESP32_Bluetooth	-42	9	0x0020 - 0x0040	26		
15	Connectable undirected - ADV_IND	D0:EF:76:45:0F:66		-43		0x0020 - 0x0040	9	0x06	
16	Scan response - SCAN_RSP	D0:EF:76:45:0F:66	ESP32_Bluetooth	-43	9	0x0020 - 0x0040	26		
17	Connectable undirected - ADV_IND	D0:EF:76:46:9B:4A		-64			21	0x06	4fafc201-1fb5-459e-8fcc-c5c9c331914b
18	Scan response - SCAN_RSP	D0:EF:76:46:9B:4A	ESP32_KAS	-64			11		
19	Connectable undirected - ADV_IND	D0:EF:76:45:0F:66		-42		0x0020 - 0x0040	9	0x06	
20	Scan response - SCAN_RSP	D0:EF:76:45:0F:66	ESP32_Bluetooth	-42	9	0x0020 - 0x0040	26		
21	Connectable undirected - ADV_IND	D0:EF:76:45:0F:66		-44		0x0020 - 0x0040	9	0x06	
22	Scan response - SCAN_RSP	D0:EF:76:45:0F:66	ESP32_Bluetooth	-44	9	0x0020 - 0x0040	26		
23	Connectable undirected - ADV_IND	D0:EF:76:46:9B:4A		-72			21	0x06	4fafc201-1fb5-459e-8fcc-c5c9c331914b
24	Scan response - SCAN_RSP	D0:EF:76:46:9B:4A	ESP32_KAS	-70			11		
25	Connectable undirected - ADV_IND	D0:EF:76:45:0F:66		-42		0x0020 - 0x0040	9	0x06	
26	Scan response - SCAN_RSP	D0:EF:76:45:0F:66	ESP32_Bluetooth	-42	9	0x0020 - 0x0040	26		
27	Connectable undirected - ADV_IND	D0:EF:76:46:9B:4A		-65			21	0x06	4fafc201-1fb5-459e-8fcc-c5c9c331914b
28	Scan response - SCAN_RSP	D0:EF:76:46:9B:4A	ESP32_KAS	-65			11		
29	Connectable undirected - ADV_IND	D0:EF:76:45:0F:66		-44		0x0020 - 0x0040	9	0x06	
30	Scan response - SCAN_RSP	D0:EF:76:45:0F:66	ESP32_Bluetooth	-44	9	0x0020 - 0x0040	26		
31	Connectable undirected - ADV_IND	D0:EF:76:46:9B:4A		-63			21	0x06	4fafc201-1fb5-459e-8fcc-c5c9c331914b

ble_packet_table

REFERENCES

BlueSnarf. (n.d.). trifinite.stuff Web site. Retrieved January 27, 2009, from

http://trifinite.org

/trifinite_stuff_bluesnarf.html

Bluesnarfing. (n.d.). Bluejacking Tools: The Biggest Collection of Bluetooth

Tools on the Internet Web site. Retrieved January 27, 2009, from

http://www.bluejackingtools.com

/bluesnarfing/Bluetooth SIG. (2008a).

How Bluetooth Technology Works. Bluetooth.com

Web site. Retrieved January 6, 2009, from

http://www.bluetooth.com/Bluetooth/Technology/Works/

Bluetooth SIG. (2008b). Security. Bluetooth.com Web site. Retrieved January

6, 2009, from

http://www.bluetooth.com/Bluetooth/Technology/Works/Security/

D3scene. (2008, April 30). BTInfo. Retrieved January 29, 2009, from

http://www.d3scene.com

/forum/general-mp/13279-btinfo.html

E-Stealth.com. (2008). Ultimate Bluetooth Mobile Phone Spy Software User

Manual. Retrieved January 29, 2009, from

http://www.jamsa.us/inventory/UltimateMobilePhoneSpyManual.pdf

Getjar. (2008, March 10). STM Bluetooth Software and Tools. Retrieved

January 29, 2009, from http://www.getjar.com/products/8042/STMBlueS

(Philippe Biondi) scapy was created in 2003. While an exact day isn't publicly documented, community sources confirm the project's inception in that year.documentation shows a copyright spanning 2008–2025, with a published build date of July 15, 2025

Biondi, P. (n.d.). Scapy: Interactive packet manipulation [Python library]. Retrieved July 16, 2025, from

https://www.secdev.org/projects/scapy/

Harvey, I. (2013, October 19). bluepy: Python interface to Bluetooth LE on Linux [GitHub repository]. Retrieved July 16, 2025, from https://github.com/IanHarvey/bluepy

BlueZ Project. (2021, April 15). btmon [GitHub wiki page]. Retrieved July 16, 2025, from https://github.com/bluez/bluez/wiki/btmon